Kubernetes - Từ nền tảng đến tự vận hành một ứng dụng

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu này giả định bạn đã biết tiến trình Linux, cổng TCP/HTTP, image/container Docker và cách đọc YAML cơ bản. Nếu chưa biết, đọc các tập tương ứng trong thư viện trước. Sau khi học, bạn cần tự trả lời được: Kubernetes giải bài toán gì, request đi qua những đối tượng nào, vì sao Pod được tạo hoặc bị thay, cách kiểm chứng một rollout và cách khoanh vùng lỗi mà không đoán mò.

Giới hạn của lab: ví dụ dùng cluster local và Nginx làm web server. Lab dạy cơ chế Kubernetes; nó không chứng minh bạn đã vận hành cluster production nhiều zone, storage doanh nghiệp hoặc upgrade control plane. Những chỗ ghi *kết quả dự kiến* là điều bạn cần tự kiểm tra trên máy, không phải output đã quan sát thay bạn.

1. Vấn đề Kubernetes giải quyết

Nếu bạn chạy ứng dụng bằng một container trên một máy, bạn phải tự xử lý khi container chết, máy hỏng, cần thêm bản sao, đổi phiên bản hoặc chuyển traffic. Docker giải bài toán đóng gói/chạy container; Kubernetes bổ sung API khai báo và các controller liên tục đối chiếu trạng thái. Bạn nói "tôi muốn 3 Pod chạy image X". Kubernetes cố tạo và duy trì trạng thái đó. Từ khóa là **cố**: nếu hết CPU, registry không truy cập được hoặc cấu hình sai, trạng thái thực tế vẫn khác mong muốn. Công việc người vận hành là đọc sự khác biệt ấy.

Kubernetes cũng đưa ra mô hình chung để định tuyến traffic, truyền cấu hình, cấp quyền, gắn storage và triển khai phiên bản mới. Đổi lại, bạn phải vận hành nhiều lớp hơn. Một ứng dụng nhỏ có thể chạy tốt trên VM hoặc dịch vụ container managed. Chọn Kubernetes khi giá trị chuẩn hóa và khả năng vận hành nhiều workload vượt chi phí học, cài, bảo trì và giám sát cluster.

Hai vòng đời phải tách riêng

```
Luồng điều khiển: bạn -> kubectl/API server -> desired state trong etcd
                                  -> controller/scheduler -> kubelet -> Pod

Luồng dữ liệu:    client -> LB/Ingress -> Service VIP/proxy -> Pod -> DB
```

EndpointSlice là dữ liệu khám phá backend cho cơ chế Service; gói tin không "đi qua" object API này.

kubectl apply thành công thuộc luồng điều khiển: API đã nhận khai báo. Nó chưa chứng minh luồng dữ liệu phục vụ người dùng. Khi một trang web lỗi, hãy vẽ hai luồng rồi đánh dấu bằng chứng ở mỗi mũi tên. Đây là mô hình quan trọng nhất của cả tập.

2. Thành phần cluster, không học thuộc tên rời rạc

API server là cửa vào để đọc/thay đổi đối tượng Kubernetes. **kubectl**, controller và nhiều công cụ khác nói chuyện qua API này. **etcd** giữ trạng thái cluster; mất quorum hoặc backup etcd kém có thể làm control plane không hoạt động đúng. **Controller manager** chạy các vòng lặp kiểm tra desired-vs-actual, ví dụ Deployment cần thêm Pod. **Scheduler** chọn node cho Pod chưa được gán dựa trên requests/ràng buộc. Trên mỗi node, **kubelet** theo dõi Pod được gán, yêu cầu container runtime chạy container và báo trạng thái lại API.

Ví dụ bạn tạo Deployment **replicas: 2**. API chấp nhận đối tượng; Deployment controller tạo ReplicaSet, ReplicaSet tạo hai Pod; scheduler chọn node; kubelet kéo image và chạy container; readiness probe quyết định Pod đã có thể nhận request. Nếu hai Pod đều Pending, đừng sửa Service trước: request

chưa có backend. Nếu hai Pod Ready mà người dùng vẫn 503, cần xem Service, Ingress và đường mạng.

Một node có thể là VM hoặc máy vật lý. Ba control-plane VM đặt trên cùng một host vật lý vẫn có failure domain chung. Ba Pod trên một node cũng không chống mất node. Tính sẵn sàng cao phải gắn với node, zone, rack, datastore và đường mạng thật.

Tự kiểm tra nhanh

- Nếu API server không truy cập được nhưng các Pod đang chạy, traffic hiện tại có thể còn đi qua không? Điều gì sẽ khó thực hiện?
- Vì sao **kubectl apply** trả thành công mà Pod vẫn **ImagePullBackOff**?
- Scheduler quyết định *nơi* chạy; thành phần nào thực sự khởi động container trên node?

Đáp án định hướng: luồng dữ liệu có thể tiếp tục một thời gian tùy thành phần khác, nhưng tạo/thay resource và điều khiển cluster bị ảnh hưởng; API chỉ nhận khai báo, việc kéo image diễn ra sau; kubelet phối hợp container runtime trên node.

3. Object model: Pod, ReplicaSet, Deployment và Service

Pod là đơn vị được lập lịch. Pod có một hoặc nhiều container chia sẻ network namespace và một số volume. Container trong cùng Pod có thể nói chuyện qua **localhost**; hai Pod khác nhau không nên dùng IP cố định vì Pod được thay có thể nhận IP mới. Hãy xem Pod là một instance có thể mất, không phải server lâu dài để SSH vào và sửa tay.

ReplicaSet giữ một số Pod phù hợp selector. **Deployment** quản lý ReplicaSet và chiến lược cập nhật cho workload stateless. Thường bạn khai báo Deployment, không tự tạo Pod riêng lẻ cho app. Khi đổi Pod template (image, biến môi trường, resource...), Deployment tạo revision/ReplicaSet mới và dần thay Pod cũ. Số **replicas: 3** là mong muốn, không phải bảo đảm ba bản sao đang phục vụ.

Service tạo tên/địa chỉ ổn định ở trong cluster và chọn backend bằng label selector. **EndpointSlice** biểu diễn tập backend hiện có cho Service. Nếu Service selector **app: notes-web** còn Pod có label **app: notes-api**, Service không chọn được Pod dù Pod Running. Nếu Pod chưa Ready, nó thường không được đưa vào backend nhận traffic thông thường. Từ Kubernetes hiện hành, nên kiểm tra EndpointSlice; lệnh **kubectl get endpoints** dựa trên API Endpoints cũ đang bị deprecate.

Ingress mô tả route HTTP/HTTPS từ ngoài vào Service, nhưng cần ingress controller triển khai. **Gateway API** là mô hình resource khác cho routing; không phải chỉ tạo YAML là có proxy. Cloud load balancer, ingress controller, Service và Pod là bốn lớp khác nhau. Khi lỗi 503, xác định lớp nào tạo mã lỗi trước khi đổi cấu hình.

Một ví dụ port để không nhầm

Giả sử Service nghe **port: 8080**, **targetPort: 80**, container thực sự nghe TCP 80. Client trong cluster gọi **notes-web:8080**; Service chuyển tới Pod:80. Trường **containerPort: 80** mô tả cổng dự kiến cho công cụ/người đọc, không tự làm ứng dụng mở cổng. Nếu Nginx trong container chỉ nghe 8081, sửa **containerPort** mà không sửa process/Service sẽ không chữa được lỗi.

4. Đọc manifest như một hợp đồng

Manifest **lesson/labs/kubernetes-local/web.yaml** trong phụ lục có hai document: Deployment và Service. Trước khi chạy, tự tìm năm mối quan hệ: **metadata.name** định danh object; **spec.selector.matchLabels** của Deployment phải khớp **template.metadata.labels**; Service selector phải khớp label Pod; **targetPort** phải tới đúng cổng process nghe; probe phải gọi endpoint app có thật.

apiVersion và **kind** xác định loại resource. **metadata** trả lời "đối tượng là ai/ở đâu". **spec** trả lời "muốn nó thế nào". **status** do hệ thống cập nhật để mô tả điều đã quan sát. Một lỗi phổ biến là nhìn **spec.replicas=2** rồi kết luận có hai Pod sẵn sàng, trong khi **status.availableReplicas** có thể bằng 0.

Nhãn (label) dùng cho selector và phân nhóm. Annotation là metadata không dùng như selector thông thường, có thể chứa thông tin release hoặc điều khiển công cụ khác. Không đổi label trong selector Deployment đang hoạt động một cách tùy tiện; nhiều selector bất biến hoặc có tác động lớn tới ownership. Trong lab lỗi Service, ta chỉ đổi **selector của Service** rồi khôi phục, tránh phá ownership của Deployment.

Image **nginx:stable** trong manifest dùng để học local, nhưng tag đó có thể trỏ tới nội dung mới theo thời gian. Bản release cần lưu image digest hoặc cơ chế pin version đủ mạnh để staging và production chạy đúng cùng artifact. Nếu image được kéo từ registry private như Harbor, cần chứng minh registry/DNS/TLS/credential hoạt động ở **node nơi Pod được lập lịch**, không chỉ trên laptop của bạn.

5. Scheduling và tài nguyên: tại sao Pod Pending?

Scheduler dùng **requests** và các ràng buộc để chọn node. **requests.cpu: 250m** nghĩa là một phần tư CPU theo đơn vị Kubernetes; **requests.memory: 128Mi** là mức bộ nhớ scheduler phải tính khi đặt Pod. Nếu node có CPU đang rảnh nhưng tài nguyên đã được **request** gần hết, Pod mới vẫn có thể Pending. Ngược lại, một Pod request thấp nhưng thực dùng cao có thể làm node quá tải. Vì vậy cần xem cả requested capacity lẫn usage thực tế.

limits.cpu có thể dẫn đến throttling khi container muốn dùng hơn trần. **limits.memory** quá thấp có thể khiến tiến trình bị OOMKilled; kernel/runtime không thể "throttle" memory giống CPU. Nếu thấy **OOMKilled**, hỏi số liệu dùng memory trước lúc chết, leak, giới hạn, burst và số replica. Tăng limit có thể là bước giảm tác động tạm thời, không thay việc tìm nguyên nhân.

Ngoài tài nguyên, Pod còn có thể Pending vì node selector/affinity không khớp, taint chưa có toleration, volume chưa bind hoặc quota namespace. Đọc **kubectl describe pod** và phần Events; event **Insufficient memory** khác hoàn toàn **FailedMount**. Đừng tăng replica khi một Pod đã không được đặt chỗ: bạn chỉ tạo thêm Pod Pending.

Ví dụ tính capacity

Một node còn 600 MiB memory có thể cấp theo scheduler. Mỗi Pod mới request 256 MiB. Về phép cộng đơn giản, chỉ hai Pod có thể được đặt chỗ (512 MiB), Pod thứ ba thiếu 168 MiB. Nhưng còn DaemonSet, overhead, phân vùng tài nguyên hệ thống và ràng buộc khác; phép tính chỉ là dự đoán ban đầu. Kiểm tra **kubectl describe node** và event để biết quyết định thực tế. Trong phỏng vấn, trình bày cả phép tính lẫn lý do nó chưa đủ.

6. Probe: ba câu hỏi khác nhau

Startup probe: ứng dụng đã hoàn tất quá trình khởi động chưa? Khi startup probe còn thất bại, kubelet chưa chạy liveness/readiness như thông thường. Dùng khi app cần warm-up lâu; cấu hình đủ thời gian nhưng không vô hạn.

Readiness probe: Pod hiện có nên nhận traffic mới không? Khi fail, Pod có thể vẫn Running nhưng không Ready; EndpointSlice phản ánh backend không sẵn sàng cho Service. Readiness có thể phụ thuộc điều kiện cần để phục vụ request, nhưng nếu tất cả Pod cùng phụ thuộc một DB đang chập chờn, loại toàn bộ Pod khỏi traffic có thể gây gián đoạn hoàn toàn. Cần quyết định theo chức năng: API chỉ đọc từ cache có còn phục vụ được không, hay mọi request đều phải ghi DB?

Liveness probe: container có đang kẹt đến mức cần restart không? Nếu liveness gọi một dependency ngoài Pod và dependency ấy chậm, kubelet có thể restart hàng loạt Pod đang khỏe về mặt tiến trình, làm quá tải thêm. Một liveness tốt thường rẻ, ổn định và chỉ fail khi restart thật sự có ích. Nếu tiến trình tự thoát khi lỗi nghiêm trọng, đôi khi chưa cần liveness riêng.

Ví dụ API mất 20 giây nạp cache: startup probe cho 30-60 giây, readiness chỉ qua khi có thể nhận request, liveness chỉ phát hiện deadlock/process không tiến triển. Các con số phải dựa trên thời gian khởi động đã đo, không chép từ tutorial. Cấu hình **periodSeconds**, **timeoutSeconds**, **failureThreshold** xác định cửa sổ phát hiện; nếu timeout 1 giây mà endpoint đôi lúc 1,2 giây dưới tải, bạn sẽ tạo false failure.

Câu hỏi suy luận

Pod **Running** nhưng **READY 0/1**, Service có 0 backend. Bạn sẽ xem gì? Trước hết **kubectl describe pod** để xem readiness failure, rồi gọi endpoint/probe path từ đúng network và đọc app log. Đổi liveness không giải quyết trực tiếp readiness. Nếu **RESTARTS** tăng liên tục, xem log **--previous** và sự kiện liveness/OOM; lỗi có thể khác.

7. Rollout: thay phiên bản không gián đoạn là một điều kiện cần chứng minh

Deployment RollingUpdate dùng **maxSurge** (được tạo thêm tạm thời) và **maxUnavailable** (có thể thiếu tạm thời) để kiểm soát tốc độ thay Pod. Nếu chỉ có một replica và **maxUnavailable** cho phép 1, có thể có khoảng không có Pod sẵn sàng. Nếu quota/node không còn chỗ cho surge, Pod mới Pending và rollout đứng. Nếu readiness quá nông, Pod mới được nhận traffic trước khi chức năng thật sẵn sàng.

Trình tự kiểm chứng một release: biết commit và image digest; so manifest trước-sau; apply qua quy trình; xem **kubectl rollout status**; so số desired/updated/available; gọi smoke test qua đường người dùng; theo dõi error rate/latency; ghi mốc thời gian; quyết định tiếp tục hoặc rollback. **rollout status** chỉ kiểm tra điều controller có thể thấy. Nginx trả **/** 200 không chứng minh Notes API ghi database được.

kubectl rollout undo có thể khôi phục Pod template của revision trước, nhưng không đảo ngược database migration, secret rotation hay cấu hình ngoài Deployment. Với schema DB, dùng kiểu mở rộng rồi thu hẹp: thêm cột/tính năng tương thích trước, deploy app mới, xác nhận, sau đó mới xóa đường cũ ở release khác. Nếu dữ liệu đã đổi không tương thích, cần plan riêng thay vì hứa rollback image trong vài giây.

Bài tính nhỏ

Deployment có 4 replica, **maxSurge: 1**, **maxUnavailable: 1**. Trong rollout, tổng Pod có thể tăng tối đa 5 và số available có thể giảm xuống 3 theo giới hạn. Nhưng điều này phụ thuộc Pod mới có thể schedule/Ready và Pod cũ vẫn khỏe. Hãy giải thích vì sao còn cần capacity cho Pod thứ năm. Nếu chỉ dùng 2 node cùng một rack, rollout không chữa được lỗi mất rack.

8. Config, Secret, identity và ranh giới bảo mật

ConfigMap cho cấu hình không nhạy cảm. Kubernetes Secret là resource khác nhưng không nên hiểu là "đã mã hóa an toàn ở mọi nơi": giá trị base64 chỉ là encoding; cần RBAC, encryption at rest theo cấu hình cluster, audit, chính sách truy cập và cách đưa Secret vào Pod. Người có quyền đọc Secret qua API hoặc quyền exec vào container có thể thấy dữ liệu tùy thiết kế. Tránh commit Secret rõ chữ vào Git.

ServiceAccount là danh tính workload khi nói chuyện với Kubernetes API. Phần lớn API web không cần đọc Pod hay sửa Deployment, nên không cấp quyền API rộng chỉ để hết lỗi **Forbidden**. Role/RoleBinding giới hạn quyền trong namespace; ClusterRole/ClusterRoleBinding có thể rộng hơn, cần đọc scope cẩn thận. **kubectl auth can-i** giúp kiểm tra một hành động cụ thể, nhưng nên thử với identity đúng của workload/người dùng.

NetworkPolicy kiểm soát traffic khi plugin mạng hỗ trợ. Không có policy không mặc nhiên cô lập namespace. Nếu áp default-deny rồi chỉ cho API tới DB, đừng quên DNS nếu app cần phân giải tên. Với mỗi đường, ghi nguồn, đích, cổng, giao thức và chiều ingress/egress. Một Pod không tới DB sau policy có thể do DNS bị chặn, rule egress thiếu, DB không lắng nghe hoặc credential sai; policy chỉ là một lớp.

SecurityContext giúp giảm quyền trong container: non-root, drop capability, filesystem read-only theo khả năng app, không privileged nếu không cần. Kiểm tra image thực dùng có chạy được với non-root và nơi app ghi file. Các chính sách này giảm rủi ro nhưng không thay vulnerability management, secret rotation và quan sát hành vi.

9. Storage: khi Pod mất thì dữ liệu đi đâu?

Filesystem writable của container gắn với vòng đời container; không nên là nguồn dữ liệu nghiệp vụ. **emptyDir** sống cùng Pod và mất khi Pod bị xóa/thay; hữu ích cho cache/tạm, không cho database cần bền. PersistentVolumeClaim yêu cầu storage từ cluster; StorageClass/provisioner cấp volume thực tùy môi trường. Cần biết access mode, vùng đặt storage, reclaim policy, backup/restore và điều gì xảy ra khi Pod chuyển node.

StatefulSet cho danh tính Pod/volume ổn định hơn Deployment, nhưng không tự cung cấp replication, backup, failover và consistency cho database. Nếu Notes API dùng SQLite file trên một Pod, scale lên ba Pod với mỗi Pod một file sẽ có ba bộ dữ liệu khác nhau; gắn chung file qua một volume tùy ý cũng có thể vi phạm giả định đồng thời/locking của ứng dụng và storage. Với nhiều replica API, đưa state vào database service được thiết kế cho chia sẻ, rồi đánh giá HA/backup của database riêng.

Câu hỏi thiết kế

Ba Pod API đều Ready, DB một instance bị hỏng. Kubernetes có "self-heal" ứng dụng không? Nó có thể thay Pod nếu Pod chết; nó không tự phục hồi dữ liệu DB hoặc sửa kết nối. Readiness API có thể fail nếu chức năng không còn phục vụ, nhưng khi tất cả Pod fail, Service có 0 backend. Bạn cần runbook cho DB, SLO, backup/replication và một quyết định rõ về việc trả lỗi hay phục vụ chế độ suy giảm.

10. Lab 1 - Tạo đường request đầu tiên trên cluster local

Điều kiện: Docker hoặc runtime phù hợp đã chạy, **kind** và **kubectl** được cài theo hướng dẫn chính thức, máy đủ tài nguyên cho một cluster local. Bài này không tạo cloud resource. Mở terminal ở thư mục **lesson/labs/kubernetes-local**. Các lệnh dưới đây là lệnh mẫu; đường dẫn và context trên máy bạn cần kiểm tra trước khi thực hiện.

Bước 0: kiểm tra đích

```
kind create cluster --name devops-study
kubectl config current-context
kubectl get nodes -o wide
```

Kết quả dự kiến: context có tên liên quan **kind-devops-study**, node ở trạng thái Ready sau khi cluster khởi động xong. Nếu context khác, dừng trước khi apply. Nếu kind đã có cluster cùng tên, xem **kind get clusters** và dùng cluster riêng đã tồn tại; không tạo trùng hoặc xóa cluster không phải của bạn.

Bước 1: dự đoán rồi áp manifest

Trước khi chạy, đọc **web.yaml** và viết ra: tên Deployment, số replica mong muốn, image, label Pod, selector Service, cổng Service và cổng container. Sau đó:

```
kubectl create namespace devops-lab
kubectl apply -n devops-lab -f web.yaml
kubectl get deployment,pods,service -n devops-lab -o wide
kubectl get endpointslices -n devops-lab -l kubernetes.io/service-name=notes-web
```

Kết quả dự kiến: Deployment mong 2 replica; sau khi image được tải và probe qua, Pod Ready; Service có EndpointSlice chứa backend. Không ghi sẵn IP/Pod name vào bài làm vì chúng thay đổi theo máy. Nếu Pod chưa Ready, dùng **kubectl describe pod -n devops-lab <ten-pod>** và đọc Events. Nếu lệnh EndpointSlice trả rỗng, kiểm tra Service selector/Pod label và readiness trước khi sửa cluster DNS.

Bước 2: truy cập qua Service

Ở terminal thứ nhất chạy **kubectl port-forward -n devops-lab service/notes-web 8080:80**. Ở terminal thứ hai gọi **curl http://127.0.0.1:8080/** hoặc **Invoke-WebRequest http://127.0.0.1:8080/** trên PowerShell. **Kết quả dự kiến:** trang mặc định Nginx, HTTP 200. Lệnh port-forward chỉ chạy khi terminal thứ nhất còn mở; Ctrl+C dừng nó. Cổng 8080 nằm trên máy bạn; số 80 bên phải là Service port. Nếu 8080 bận, chọn 8081:80.

Bước 3: ghi bằng chứng

Trong nhật ký lab, lưu context, namespace, manifest đã dùng, trạng thái Pod/Deployment, số EndpointSlice/backend, HTTP status và thời gian. Viết một câu giải thích từng mũi tên: máy bạn -> port-forward -> Service -> Pod -> Nginx. Nếu thấy 200 nhưng không biết Pod nào nhận request, thêm log/request ID hoặc thử scale xuống một Pod để suy luận; đừng đoán từ tên Service.

11. Lab 2 - Ba lỗi có chủ đích và cách chữa bằng bằng chứng

Chỉ làm khi Lab 1 đã hoạt động. Trước mỗi lỗi, lưu **web.yaml** gốc và giữ lệnh khôi phục. Không thử trên cluster dùng chung. Mục tiêu không phải làm hỏng cho vui, mà học phân biệt triệu chứng **Pod**, **Service** và **probe**.

Lỗi A: Service selector sai

Trong bản sao manifest, đổi chỉ **spec.selector.app** của **Service** từ **notes-web** thành **wrong-label**, rồi apply trên **devops-lab**. Không đổi Deployment selector. Kiểm tra:

```
kubectl get pods -n devops-lab --show-labels
kubectl get service notes-web -n devops-lab -o yaml
kubectl get endpointslices -n devops-lab -l kubernetes.io/service-name=notes-web
```

Dự kiến: Pod vẫn có thể Ready, nhưng Service không có backend Ready phù hợp; port-forward qua Service hoặc request qua đường Service thất bại. Nếu còn EndpointSlice cũ trong khoảnh khắc đầu, chờ controller reconcile rồi đọc lại. Kết luận: lỗi ở ánh xạ label/selector, không phải image. Khôi phục Service selector từ manifest gốc, apply và xác nhận backend trở lại.

Lỗi B: readiness path không tồn tại

Trong bản sao manifest, đổi **readinessProbe.httpGet.path** thành **/missing**, giữ liveness path **/**. Apply rồi quan sát **kubectl get pods -w**, **kubectl describe pod** và EndpointSlice. **Dự kiến:** container có thể Running nhưng **READY** giảm; Events cho biết readiness probe trả mã lỗi; Service không gửi traffic bình thường tới Pod chưa Ready. Nếu rollout còn Pod cũ, bạn có thể vẫn thấy traffic tới Pod cũ, vì Deployment giữ khả dụng theo chiến lược. Đây là điểm cần quan sát chứ không áp sẵn một kết quả tuyệt đối. Khôi phục path **/**, chờ Ready, kiểm tra lại request.

Lỗi C: image tag không tồn tại

Chỉ trong lab, dùng bản sao manifest đổi image thành tag cố ý sai, ví dụ **nginx:tag-khong-ton-tai-devops-lab**. Apply, xem **kubectl describe pod**, Events và **kubectl rollout status**. **Dự kiến:** Pod mới không kéo được image (**ErrImagePull**/**ImagePullBackOff**); Deployment có thể giữ Pod cũ để phục vụ tùy strategy/capacity. Đọc thông báo cụ thể: **not found** khác **unauthorized**, TLS lỗi hoặc DNS lỗi. Khôi phục manifest gốc, apply và quan sát rollout đến khi Pod Ready.

Mẫu nhật ký cần nộp

Lỗi	Triệu chứng người dùng	Bằng chứng trong cluster	Giả thuyết đã loại trừ	Sửa	Bằng chứng phục hồi
Selector sai	Service không tới backend	Pod Ready, EndpointSlice không có backend phù hợp	Image vẫn chạy	Khôi phục selector	EndpointSlice có backend, HTTP 200
Readiness sai	Traffic giảm/lỗi khi rollout	Probe failure, Ready false	Process vẫn có thể Running	Sửa path	Ready true, request qua
Image sai	Pod mới không lên	Pull event và rollout kẹt	Không do Service selector	Pin image đúng	Rollout hoàn tất, smoke qua

Điền giá trị thực tế thay cho mô tả dự kiến. Nếu môi trường của bạn cho kết quả khác, giữ bằng chứng và giải thích vì sao; đó mới là học vận hành.

12. Troubleshooting theo thứ tự, không "restart thử"

- **Xác nhận tác động:** endpoint nào, từ lúc nào, bao nhiêu request, bản release nào, tất cả hay một phần người dùng?
- **Đọc trạng thái mong muốn và thực tế:** Deployment desired/available, Pod phase/Ready/restarts, Service selector, EndpointSlice, Ingress route.
- **Đọc Events và log:** scheduler, image pull, mount, probe, app exception. Dùng **kubectl logs --previous** nếu container đã restart.
- **So thay đổi gần nhất:** image digest, config/secret, chart values, network policy, node/registry/storage.
- **Lập giả thuyết và phép thử nhỏ:** một biến, read-only trước; nếu cần giảm tác động, dùng rollback/route theo runbook.
- **Xác nhận phục hồi:** request thật, error rate, latency, backlog, thời gian ổn định; ghi mốc.

Bảng triệu chứng -> lệnh đọc đầu tiên

Triệu chứng	Đọc đầu tiên	Nhánh điều tra tiếp
Pending	**kubectl describe pod** + Events	resource, taint, affinity, PVC, quota
ImagePullBackOff	**kubectl describe pod**	tag/digest, registry, TLS, DNS, pull secret
CrashLoopBackOff	**kubectl logs --previous**	exit code, config, OOM, liveness, dependency
Running 0/1	**kubectl describe pod**	readiness path/port, startup, app log
Service 0 backend	selector/labels + EndpointSlice	readiness, namespace, port
HTTP 502/503	ingress log + Service backend	upstream, route, port, Pod Ready
Chậm, CPU thấp	trace/DB/queue, resource metric	lock, I/O, connection pool, DNS timeout

Trong cluster thật, nhiều nguyên nhân có thể cùng tồn tại. Bảng chỉ chọn *điểm bắt đầu có xác suất hữu ích cao*, không phải công thức kết luận.

13. HA, autoscaling và vận hành production

HPA thay số Pod theo metric/target và điều kiện cấu hình. Nếu HPA dùng CPU utilization, mẫu số dựa vào CPU request. Ví dụ Pod request 250m và dùng 150m thì khoảng 60% của request. Nếu DB là nút thắt, thêm Pod có thể làm connection tới DB tăng và hệ tệ hơn. Nếu node thiếu capacity, Pod mới Pending; cluster autoscaler (khi có) giải tầng node, nhưng cần thời gian và quota. Tự động scale phải có min/max, load test, metric người dùng và giới hạn downstream.

PodDisruptionBudget giới hạn số Pod bị evict đồng thời trong gián đoạn **tự nguyện** như drain node; không bảo vệ trước node đột ngột chết. Topology spread hoặc anti-affinity giúp phân tán Pod qua node/zone nếu các failure domain thật sự độc lập. Với on-prem, kiểm tra VM->host->rack->datastore; với managed cluster, kiểm tra node pool/zone và phụ thuộc LB/storage. "Có ba replica" chỉ là một dòng YAML, không phải chứng nhận HA.

Upgrade cluster cần xem version skew, API deprecated, CNI/CSI, ingress controller, CoreDNS, kube-proxy/runtime và backup etcd/control plane nếu tự quản. Nâng từng lớp theo tài liệu đúng phiên bản, có staging và rollback khả thi. Không dùng lab kind một node để kết luận upgrade production nhiều node an toàn.

Bài phỏng vấn tổng hợp

Một fintech có 3 Pod API trên 3 VM Kubernetes nhưng cả 3 VM ở một host VMware; DB nằm trên một VM cùng datastore. Bạn sẽ thay gì để đáp ứng mục tiêu chịu hỏng một host? Trả lời cần xét placement VM qua host/rack, capacity N-1, storage/datastore, DB HA/replication, LB/ingress, backup/restore và diễn tập failover. Chỉ tăng replicas lên 6 không giải được failure domain chung.

14. Ca điều tra có lời giải: HTTP 503 ngay sau release

Bối cảnh: một API có 3 replica. Trước release, lỗi 5xx dưới 0,1%. Sau đổi image, ingress trả 503 cho khoảng 40% request. **kubectl apply** thành công. Người trực ca thấy 3 Pod **Running**, trong đó 2 Pod **Ready**; một Pod mới **0/1**. Đừng gắn ngay nguyên nhân cho ingress: 503 là triệu chứng, có thể do proxy không tìm thấy upstream, upstream trả lỗi, route sai hoặc các backend còn lại quá tải.

Bước 1: đóng khung sự cố. Xác nhận URL, thời điểm, tỷ lệ lỗi và revision; tách request đi qua ingress với request nội bộ qua Service. Nếu request qua Service vẫn lỗi, tập trung Service/Pod/app trước. Nếu Service tốt còn ingress lỗi, xem route, TLS, annotation và ingress-controller logs. Nếu chỉ vài client lỗi, còn phải xem DNS/LB/cache. Mỗi phép thử trả lời một câu hỏi, không chạy lệnh theo thói quen.

Bước 2: so desired với actual. Xem Deployment **replicas**, **updatedReplicas**, **availableReplicas**; xem ReplicaSet cũ/mới và Pod theo label. Ba Pod **Running** không đồng nghĩa ba backend. Kiểm tra EndpointSlice: địa chỉ nào được đánh dấu Ready? Nếu còn hai backend nhưng traffic quá tải, 503 có thể tăng ngay cả khi Service không rỗng. So giới hạn CPU/memory và latency ở hai Pod còn lại.

```
kubectl get deploy,rs,pods -n devops-lab -o wide
kubectl describe deploy notes-web -n devops-lab
kubectl get endpointslice -n devops-lab -l kubernetes.io/service-name=notes-web -o yaml
kubectl describe pod <ten-pod-moi> -n devops-lab
kubectl logs <ten-pod-moi> -n devops-lab --previous
```

--previous chỉ hữu ích nếu container đã restart; nếu chưa, đọc log hiện tại. **Giả sử** Events của Pod mới ghi readiness **HTTP probe failed with statuscode: 404**. Khi đó giả thuyết mạnh là image mới đổi đường **/health** mà manifest chưa đổi. Xác nhận bằng cách đọc config probe trong Pod hiện tại, startup log và gọi đúng endpoint từ trong cùng mạng nếu có thể. Đừng sửa thành liveness để "hết đỏ": readiness là quyền nhận request. Nếu probe đã chỉ tới đúng path nhưng 404 do app route chưa sẵn sàng, xem quá trình startup và thời gian nạp dependency.

Bước 3: giảm tác động. Nếu release mới gây lỗi và rollback image không phá tương thích dữ liệu, quay về revision đã biết tốt theo quy trình. Theo dõi **rollout status**, số EndpointSlice backend Ready, error rate và latency. Nếu schema DB đã đổi không tương thích, rollback image có thể tạo lỗi mới; cần kế hoạch roll-forward hoặc khôi phục dữ liệu đã chuẩn bị. Ghi lại điều kiện quyết định. Một sự cố production không nên được xử lý bằng sửa probe cho xanh trước khi hiểu endpoint **/health** có ý nghĩa gì.

Bước 4: ngăn lặp lại. Thêm contract test cho health endpoint; CI render manifest và so đường probe với app; staging test với traffic thật và ước lượng capacity khi mất một Pod; canary hoặc progressive rollout cho thay đổi rủi ro. Postmortem cần nói rõ bằng chứng nào giúp tìm nguyên nhân, thời gian phát hiện và vì sao guardrail cũ bỏ sót. "Dev quên sửa YAML" là mô tả hành động, chưa phải phân tích hệ thống.

Vì sao không kết luận quá sớm?

Nếu EndpointSlice có đủ 3 backend nhưng ingress vẫn 503, giả thuyết probe 404 không giải thích toàn bộ lỗi. Có thể ingress đang trỏ Service khác, sai port, hoặc upstream trả 503 theo nghiệp vụ. Nếu chỉ Pod trên một node lỗi, xem node network/CNI/DNS/volume. Một lời giải tốt có thể thay khi bằng chứng mới đến; vận hành là quá trình cập nhật giả thuyết.

15. DNS và mạng Pod: đường đi cần hiểu trước khi đụng CNI

Pod có IP riêng; Service có địa chỉ/tên ổn định và cơ chế chuyển tới các backend. Pod gọi **notes-web.devops-lab.svc.cluster.local** thì DNS trong cluster phân giải tên Service; resolver trong Pod thường dùng search domains nên tên ngắn **notes-web** có thể hoạt động khi cùng namespace. DNS phân giải thành công chỉ chứng minh bạn tìm được địa chỉ, chưa chứng minh TCP tới Pod, TLS hợp lệ hay ứng dụng trả đúng. Nếu **curl** theo Service lỗi, thử tách: DNS lookup, TCP connect, HTTP response, rồi log app. Đừng thay CoreDNS vì một Service không có endpoint.

CNI là lớp cung cấp network cho Pod và thực thi một phần chính sách tùy plugin; kube-proxy hoặc implementation khác xử lý Service forwarding tùy cluster. Một NetworkPolicy YAML được API chấp nhận nhưng cluster dùng plugin không thực thi policy thì kết quả kiểm soát mạng có thể không như mong muốn. Ngược lại, default-deny egress có thể chặn DNS khiến app báo "database host not found", trong khi DB khỏe. Kiểm tra plugin, policy thực tế, namespace selector và traffic mẫu từ đúng Pod.

Phân biệt ba bài kiểm tra: **kubectl port-forward** có thể đi theo đường khác traffic người dùng qua LB/Ingress; request từ Pod nội bộ qua Service kiểm tra cluster DNS và backend; request từ ngoài qua domain/TLS kiểm tra toàn bộ biên. Một phép thử thành công ở lớp thấp không chứng minh lớp cao. Khi viết runbook, ghi rõ điểm bắt đầu và điểm kết thúc của mỗi phép thử.

16. Câu hỏi kiểm tra có lời giải

Câu 1. Tạo Deployment **replicas: 3**, cả ba Pod **Running**, nhưng Service không nhận request. Bạn kiểm tra theo thứ tự nào? **Lời giải:** xem **Ready** chứ không chỉ phase; xem Service selector và Pod labels; xem EndpointSlice backend; sau đó port/targetPort và endpoint app. Nếu EndpointSlice trống vì cả ba chưa Ready, tập trung probe/app. Nếu endpoint có đủ nhưng request timeout, điều tra port, network policy và process lắng nghe. Mỗi bước phân biệt được một nhóm nguyên nhân.

Câu 2. Một Pod request 100m CPU, dùng 800m; limit 500m. Node còn nhiều CPU nhưng app chậm. Vì sao? **Lời giải:** request dùng để lập lịch, limit CPU có thể gây throttling khi dùng vượt 500m. Xem metric throttling và latency theo thời gian; nếu xác nhận, điều chỉnh limit/request dựa trên tải đo được hoặc bỏ CPU limit theo chính sách tổ chức. Chỉ nhìn tổng CPU node không đủ.

Câu 3. Bạn đặt 3 replica ở 3 VM cùng host vật lý. Có chịu được hỏng một host không? **Lời giải:** không. VM khác nhau nhưng failure domain host là một. Cần phân tán VM qua host và bảo đảm capacity khi một host mất, đồng thời kiểm tra storage, DB và network LB. Số replica là điều kiện về số tiến trình, không phải topology.

Câu 4. Vì sao readiness kiểm tra DB có thể làm outage nặng hơn? **Lời giải:** nếu DB chậm ngắn hạn, mọi Pod cùng fail readiness thì Service mất backend, kể cả endpoint chỉ đọc cache có thể phục vụ. Thiết kế readiness theo khả năng phục vụ chức năng cốt lõi và chế độ suy giảm; liveness không nên phụ thuộc DB ngoài Pod. Quyết định này cần SLO và hành vi sản phẩm cụ thể.

Câu 5. Khi nào **kubectl rollout undo** chưa đủ? **Lời giải:** khi schema/dữ liệu/secret hoặc service phụ thuộc đã thay đổi không tương thích; Deployment rollback chỉ đổi Pod template. Kế hoạch release phải bao gồm migration tương thích, checkpoint backup, hoặc roll-forward được diễn tập.

17. Cổng năng lực và câu hỏi phỏng vấn

Bạn chỉ đánh dấu tập này hoàn thành khi: (1) vẽ được luồng điều khiển và luồng request; (2) chạy Lab 1 và ba lỗi Lab 2, lưu bằng chứng; (3) giải thích chính xác sự khác nhau giữa Pod Running, Ready, Deployment Available và Service có backend; (4) đưa kế hoạch rollout/rollback xét database; (5) trả lời case HA theo failure domain.

- **Vì sao Service dùng selector thay vì IP Pod cố định?** Vì Pod có thể được thay/IP đổi; controller cập nhật backend dựa trên label và readiness.
- **Readiness và liveness khác nhau thế nào?** Readiness quyết định Pod nhận traffic mới; liveness quyết định có restart container. Dùng sai có thể gây outage.
- **Pod Pending nhưng node CPU chưa đầy, vì sao?** Scheduler xét requests/ràng buộc, không chỉ usage; còn memory, taint, affinity, PVC, quota.
- **Rollout status xanh đã đủ chưa?** Chưa; cần smoke test đường nghiệp vụ và telemetry sau release, đặc biệt DB/migration.
- **HPA thêm Pod nhưng latency tăng, vì sao?** Nút thắt có thể ở DB/queue, connection tăng hoặc Pod mới Pending; phải đo toàn đường đi.

18. Dọn lab và ghi giới hạn trải nghiệm

Dừng port-forward. Kiểm tra context là **kind-devops-study** và namespace **devops-lab** chỉ chứa resource lab. Xóa namespace bằng **kubectl delete namespace devops-lab**; xác nhận bằng **kubectl get namespace devops-lab**. Chỉ nếu cluster **devops-study** do bạn tạo và không còn dùng, xóa bằng **kind delete cluster --name devops-study**. Hai lệnh xóa này phá tài nguyên tương ứng; không copy sang môi trường dùng chung. Giữ nhật ký/ảnh chụp/trích log để sau này giải thích phỏng vấn.

Nguồn chính thức (đối chiếu phiên bản trước khi chạy):

- Kubernetes Concepts: https://kubernetes.io/docs/concepts/
- Deployments và rollout: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Service và EndpointSlice: https://kubernetes.io/docs/concepts/services-networking/service/
- Probe: https://kubernetes.io/docs/concepts/configuration/liveness-readiness-startup-probes/
- Troubleshooting applications: https://kubernetes.io/docs/tasks/debug/debug-application/
- kind quick start: https://kind.sigs.k8s.io/docs/user/quick-start/

Phụ lục: mã lab để đọc offline

lesson/labs/kubernetes-local/web.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: notes-web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: notes-web
  template:
    metadata:
      labels:
        app: notes-web
    spec:
      containers:
        - name: web
          image: nginx:stable
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              cpu: 250m
              memory: 128Mi
          readinessProbe:
            httpGet:
              path: /
              port: 80
          livenessProbe:
            httpGet:
              path: /
              port: 80
---
apiVersion: v1
kind: Service
metadata:
  name: notes-web
spec:
  selector:
    app: notes-web
  ports:
    - port: 80
      targetPort: 80
```